

# Integración Active-IA · AI-Native

Está construida. Esto es lo que falta para encenderla.

De: equipo de Active-IA (api.active-ia.com)

Para: equipo de AI-Native (tutor.active-ia.com)

Fecha: 24 de agosto de 2026

Reemplaza: nuestro documento del 20/08 — cambió el estado de casi todo

**Novedad principal desde el 20/08: el endpoint de corrección existe.** Ya pueden publicar un TP con sus ejercicios y sus rúbricas, y pedir que se corrija ejercicio por ejercicio mandando el resultado de su sandbox. Los §3.1, §3.2, §3.3 y §3.4 de su pedido están implementados y probados.

Lo que sigue son cuatro definiciones que necesitamos de ustedes, dos avisos sobre el contrato y una cosa que todavía no está construida y conviene que sepan.

## 1. Lo que necesitamos de ustedes

| # | Qué necesitamos                                                                                                                                        | Qué bloquea                                                                                                                 | Urgencia    |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------|
| 1 | <b>Que marquen, por criterio, si depende de la ejecución</b> al publicar sus rúbricas. Campo booleano nuevo, opcional (ver §3.1).                      | La garantía de <i>compila: false</i> que ustedes pidieron. Está construida y probada, pero <b>sin esa marca no aplica</b> . | <b>Alta</b> |
| 2 | <b>La respuesta sobre personería, por escrito.</b> Nos confirmaron de palabra que podemos continuar y lo agradecemos; necesitamos el respaldo escrito. | El despliegue con datos de alumnos reales. No bloquea nada más.                                                             | <b>Alta</b> |
| 3 | <b>Confirmar el contrato</b> contra su doble HTTP, y una ventana para apuntar su cliente a un entorno de staging nuestro.                              | El deploy. Preferimos que una diferencia aparezca contra su doble y no contra producción.                                   | <b>Alta</b> |
| 4 | <b>¿Quién puede pedir una anonimización?</b> ¿Solo un administrador nuestro, o también su sistema desde el procedimiento automatizado?                 | El diseño del borrado por alumno (§3.6).                                                                                    | Media       |

El punto 1 es el que más nos importa y es trabajo suyo, no nuestro. Lo explicamos completo en §3.1: no es un trámite, es lo que hace que la garantía que pidieron sea una garantía.

## 2. Qué está construido

| Lo que pidieron                                        | Estado       |
|--------------------------------------------------------|--------------|
| §3.1 — nivel de ejercicio debajo del TP                | <b>Listo</b> |
| §3.2 — <i>external_ref</i> en lugar de <i>cmid</i>     | <b>Listo</b> |
| §3.3 — endpoints de escritura de TPs                   | <b>Listo</b> |
| §3.4 — corrección recibiendo el resultado de los tests | <b>Listo</b> |
| §7.2 — el 500 de <i>GET /entregas/{id}</i>             | <b>Listo</b> |

|                                              |                                                 |
|----------------------------------------------|-------------------------------------------------|
| Bugs 1, 4 y 5 del motor                      | Listo (el 4 y el 5 reducidos, ver §5)           |
| Bug 6 — recomienda lo que la cátedra prohíbe | En curso                                        |
| Bugs 2 y 3 — penalizaciones y desglose       | Análisis cerrado, pendiente de revisión interna |
| §3.5 — cuenta de servicio                    | <b>Todavía no</b> (ver §6)                      |
| §3.6 — borrado por alumno                    | Especificado, sin empezar                       |

## 2.1 Los cuatro endpoints

POST /api/v1/trabajos-practicos/

PUT /api/v1/trabajos-practicos/by-ref/{external\_ref}

GET /api/v1/trabajos-practicos/by-ref/{external\_ref}

POST /api/v1/correcciones/ejercicios/{ejercicio\_ref}/corregir

El **PUT** por referencia externa es idempotente: responde **201** cuando creó y **200** cuando actualizó. El de corrección es **sincrónico** — devuelve la nota y el desglose en la respuesta, no un 202. Un ejercicio solo entra cómodo en su timeout de 90s.

**Garantía que no pidieron pero necesitan:** el *rubrica\_id* de un ejercicio es estable de por vida. Los emparejamos por *external\_ref*, nunca por orden ni por título, así que reordenarlos o renombrarlos en su plataforma no rota ninguna asociación. Si cambiara, sus correcciones anteriores quedarían colgando de una rúbrica que ustedes ya no vinculan a ese ejercicio.

## 3. Lo que cambia de su lado

### 3.1 El campo que necesitamos que marquen

Nos pidieron que con *compila: false* no cerremos criterios del tipo «el programa funciona». Estuvimos de acuerdo, y lo hicimos **determinístico en nuestro backend**, no como una instrucción de prompt.

¿Por qué esa distinción importa? Porque ponerlo en el prompt sería repetir exactamente el bug 2 que ustedes reportaron: la rúbrica pedía una penalización del 30% y el motor aplicó 0%. De este motor ya está medido que no honra reglas declaradas en su propia rúbrica. Una garantía declarativa no es una garantía.

Pero para que sea determinística hace falta un dato que **solo la rúbrica tiene**: cuál de sus criterios necesita que el programa corra. Nuestro backend no puede inferirlo — «usó la interfaz o enumeró los tipos concretos» no depende de la ejecución; «produce la salida esperada», sí.

```
"rubrica": {  
  "criterios": [  
    { "nombre": "El programa produce la salida esperada",  
      "descripcion": "...",  
      "puntaje_max": 4,  
      "depende_de_ejecucion": true } <-- esto  
  ]  
}
```

Es **opcional y por defecto va en falso**, así que nada se rompe si no viene. Pero si no viene, el comportamiento es el de hoy y **la garantía no aplica** — no porque no queramos darla, sino porque no tendríamos con qué distinguir un criterio de diseño de uno de funcionamiento. Cuando pasa, queda registrado en nuestros logs.

### 3.2 Dos campos nuevos en la respuesta de corrección

**criterios\_sin\_ejecucion** — los identificadores de los criterios que cerramos en 0 porque el código no compilaba. Va aparte para que puedan mostrárselos distinto al docente: «no lo hizo» y «no se pudo verificar porque no compila» son dos cosas distintas y merecen leerse distinto.

**La respuesta NO trae una nota agregada del TP.** Devolvemos nota por ejercicio, como pidieron; el promedio ponderado lo hacen ustedes. Tenemos un test que verifica que ese campo no exista, para que nadie lo agregue «porque es útil».

### 3.3 Un campo opcional en el pedido de corrección

Nuestro modelo exige que toda entrega pertenezca a una comisión, y ustedes no tienen comisiones. Su documento no podía verlo porque no ve nuestro modelo de datos.

Lo resolvimos con una **comisión de integración** que configuramos una vez por materia, más un campo **opcional** *comision\_external\_ref* en el cuerpo, por si alguna vez quieren modelar cohortes.

El contrato que ya implementaron **NO cambia**. {alumno\_ref, codigo, resultado\_tests} sigue siendo válido tal cual. El campo nuevo es opcional y si no viene usamos la comisión de integración.

### 3.4 Los casos ocultos mal formados los rechazamos

Compartimos su razonamiento del §3.3.1: lo que no recibimos no lo podemos citar. Si por un error llega un caso oculto con su salida esperada o su aserción, respondemos **422 nombrando el ejercicio y el caso**, en vez de descartar el campo en silencio. Falla cuando el docente publica, que es barato, y no con un alumno esperando su corrección.

### 3.5 Límite de 50 ejercicios por TP

Su documento describe TPs de cuatro, así que hay margen de sobra. Lo decimos para que sea un límite *conocido* y no algo que descubran como un timeout.

## 4. Dos avisos

### 4.1 La clave del 409 de entregas (su §7.3)

Dicen que keyea por (*comision\_id*, *rubrica\_id*, *alumno\_nombre*). El índice único real es (***rubrica\_id***, ***alumno\_nombre***), sin *comision\_id*. Su conclusión operativa es correcta y no cambia — comparar el *rubrica\_id* no es opcional — pero como dijeron que los tres comportamientos están codificados en su cliente, preferimos que el dato esté bien.

### 4.2 En el endpoint de ejercicio no hay 409 por entrega existente

Si vuelven a corregir el mismo ejercicio para el mismo alumno, **reusamos la entrega** y guardamos la corrección anterior en nuestro historial. Reintentar es la misma llamada, sin ramificar por un conflicto.

## 5. Lo que hicimos y lo que todavía no

### *Los bugs 4 y 5 están reducidos, no eliminados*

Agregamos evidencia obligatoria por criterio —el motor tiene que citar la línea de código que respalda cada puntaje, y verificamos que esa cita exista de verdad en lo que recibió— y reglas de juicio con sus dos casos control como ejemplos. Eso sube mucho el costo de afirmar algo sin sustento, pero son reglas de juicio y no se verifican mecánicamente.

**La eliminación real de esos dos modos de fallo la da el §3.4**, que ya está implementado: un test ejecutado no se deja engañar ni por «vio las piezas» ni por un literal hardcodeado. Por eso el punto 1 de nuestro pedido importa tanto.

### *El bug 1 está arreglado, y la causa era de una línea*

La lista de archivos de la entrega existía en nuestra base y nunca llegaba al modelo. Recibía un texto concatenado y tenía que inferir qué había entregado el alumno.

## 6. La cuenta de servicio todavía no está

**Y esto los afecta para probar.** El §3.5 está especificado pero sin construir: toca autenticación y autorización, así que tiene un proceso de revisión más estricto de nuestro lado.

Mientras tanto, para las pruebas contra staging podemos darles acceso con una cuenta de coordinador acotada a las materias del piloto. No es lo ideal —caduca, no se revoca sola y en la auditoría las acciones aparecen como de una persona— pero desbloquea las pruebas sin esperar.

Cuando la cuenta de servicio esté: la credencial se entrega **una sola vez** y no es recuperable; si se pierde, se rota. Va a estar acotada a las materias del piloto y con vencimiento explícito. **¿Por qué canal quieren recibirla?** No la vamos a mandar por mail ni por chat.

## 7. Una limitación que preferimos declarar

**La anonimización va a cubrir la base viva, no los respaldos históricos.** Cuando implementemos el §3.6, el procedimiento va a destruir código, devolución y pseudónimo de la base en uso. Los respaldos anteriores a esa operación quedan fuera de su alcance.

Lo decimos porque afecta el alcance real del compromiso que ustedes firmaron con los alumnos del piloto. Si necesitan que los respaldos también queden cubiertos, hay que acordar una política de retención.

## 8. Lo que confirmamos que NO hacemos

- **No ejecutamos código.** El sandbox es de ustedes y funciona.
- **No calculamos la nota final del TP.** Nota por ejercicio; el promedio ponderado lo hacen ustedes.
- **No escribimos nada de su lado.** La integración es de una sola dirección.
- **No tocamos el flujo de Moodle.** *cmid* sigue funcionando igual.
- **No hacemos corrección en lote.** Se dispara de a un ejercicio.

---

Cierre. Su documento sigue siendo la razón por la que esto avanzó rápido: que cada afirmación viniera con fecha de verificación y caso de evidencia nos dejó ir directo a *por qué* pasaban los bugs en vez de discutir si pasaban. Los dos hallazgos que más nos sirvieron —el bug 2 no era desobediencia del modelo, y el bug 1 tenía una causa de una línea— salieron de poder buscar algo concreto.

Quedamos a la espera de los puntos 1 a 3.